

300

Years Later — guide du studio

Construire un jeu coopératif 3D réaliste sous Unreal Engine 5.8 avec une IA autonome, de A à Z : installation, code, tests, conformité, médias, landing page cinématique, lancement.

Édition du 07/10/2026 · licence CC BY 4.0

Sommaire

1. À lire d'abord
2. Demarrage rapide
3. Guide a z
4. Installation
5. Temps et couts
6. Securite et confidentialite
7. Personnaliser
8. Autres ia
9. Faq depannage
10. Schémas
11. Captures d'écran
12. Images du jeu

À lire d'abord

Ce guide ne contient aucune image du jeu tant qu'aucun rendu n'a été produit dans Unreal Engine et validé par un humain. Les captures présentées sont réelles : la landing page de référence et les sorties des outils du kit. Les schémas expliquent le fonctionnement ; ils ne représentent pas le jeu.

Le jeu visé est un titre indépendant réaliste de haute qualité, pas une production de plusieurs centaines de millions. Les budgets et durées sont détaillés dans le guide « Temps et coûts ».

Démarrage rapide (10 minutes)

 [English version](#)

Ce que vous obtenez

Un **studio de jeu vidéo piloté par IA** : la conception complète d'un jeu coop 3D réaliste (*300 Years Later*), un corpus juridique, une landing page cinématique, des tests, et surtout des **skills** qui permettent à Claude Code (ou à une autre IA) de construire le jeu dans Unreal Engine 5.8, en autonomie de A à Z, en ne vous sollicitant qu'aux décisions qui vous reviennent.

Ce dont vous avez besoin

- Un PC Windows 10/11 avec une carte graphique récente (RTX 3070/4070 ou mieux), 32 Go de RAM, 300 Go libres — détails dans [02_INSTALLATION.md](#).
- Claude Code (ou un autre agent de code) et un compte Epic Games.
- Du temps et un budget réalistes : lisez [03_TEMPS_ET_COUTS.md](#) avant de vous lancer.

Les 5 premières commandes

```
git clone <URL-DE-CE-DÉPÔT> 300-years-later && cd 300-years-later
python3 tools/doctor.py           # ce qui est installé / ce qui manque
python3 tools/validate_data.py    # les données du jeu sont cohérentes
python3 tools/privacy_scan.py    # aucune donnée personnelle ni secret
claude                           # lance Claude Code dans le dossier
```

Puis tapez dans Claude Code :

Lis AGENTS.md puis applique le skill game-studio. Travaille en autonomie de A à Z selon studio.config.yaml ; ne me sollicite qu'aux arrêts obligatoires, en regroupant tes questions avec ta recommandation. Réponds-moi en français. *(facultatif)* Instructions en plus : ...

Vous préférez garder la main ? Mettez autonomy.level: guided dans studio.config.yaml (l'IA demande avant chaque étape). Vous voulez qu'elle installe aussi sans demander ? full.

Régler l'IA une fois pour toutes : studio.config.yaml

Clé	Effet
autonomy.level	guided · autonomous (défaut) · full
autonomy.max_questions_per_session	nombre maximal de questions par session (défaut 3)
project.languages	langues du jeu et des textes publics (défaut fr, en)
defaults.*	scénario d'équipe, budget assets mensuel, direction artistique, mode de la landing...
extra_instructions	vos consignes libres, appliquées à chaque session

Votre message l'emporte toujours sur ce fichier.

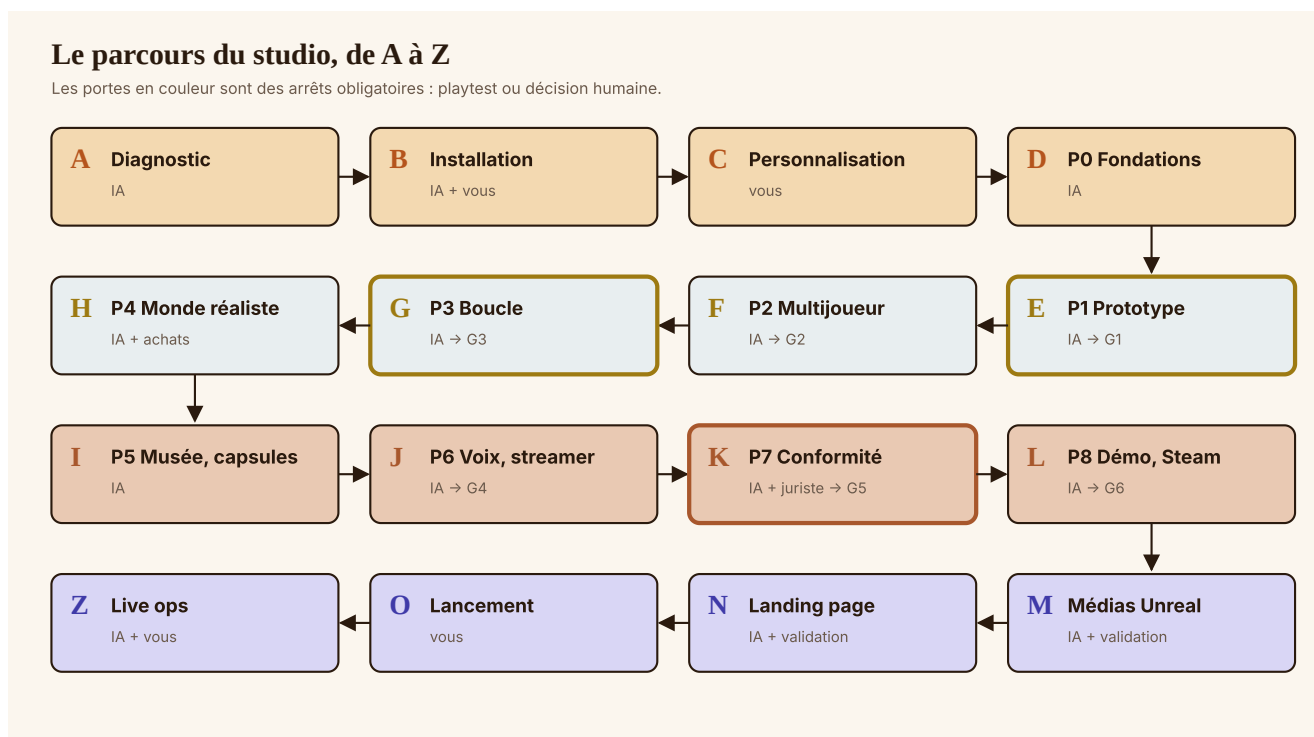
Ensuite

L'IA avance seule et tient trois fichiers à jour : STUDIO_STATE.md (où on en est), DECISIONS.md (ce qu'elle a décidé seule, réversible), QUESTIONS.md (ce qu'elle attend de vous, avec sa recommandation — répondez « ok » pour l'accepter). Les arrêts obligatoires (comptes, installation d'Unreal, achats, publication...) sont décrits dans le [guide de A à Z](#).

Guide de A à Z

🇬🇧 [English version](#)

Ce guide décrit le parcours complet, du dépôt cloné au jeu en accès anticipé, tel que le pilote le skill game-studio. Pour chaque étape : **qui fait quoi**, **ce que vous devez vérifier**, et **la durée indicative** (scénario « studio réduit » de [03_TEMPS_ET_COUTS.md](#)).



Légende des rôles

- **IA** : Claude Code (ou un autre agent) appliquant les skills, en mode autonome par défaut (studio.config.yaml).
- **Vous** : le porteur du projet, qui décide, achète, valide et publie.
- **Pro** : un professionnel externe (juriste, artiste, compositeur...).

A. Diagnostic — 1 heure

IA lance `tools/doctor.py`, vérifie le matériel, lit `STUDIO_STATE.md`. **Vous** lisez le rapport. Si le PC est insuffisant, l'IA peut quand même avancer la conception, la landing page et le juridique.

B. Installation — 1 à 3 jours (téléchargements longs)

IA affiche le plan d'installation (`install_windows.ps1 -Plan`), puis l'exécute après un accord global unique (sans demander en mode `full`). Le plugin Epic et `frontend-design` sont déjà déclarés dans `.claude/settings.json`. **Vous** créez le compte Epic, installez Unreal Engine 5.8 depuis l'Epic Games Launcher, installez le plugin officiel Epic dans Claude Code, activez le serveur MCP dans l'éditeur. Détails : [02_INSTALLATION.md](#). **Vérification** : `doctor.py` tout vert ; l'IA liste les acteurs de la carte ouverte via MCP.

C. Personnalisation — 1 jour

IA lit `studio.config.yaml` ; **vous** n'intervenez que si le fichier est vide ou si vous voulez changer le titre public, les langues, les plateformes ([05_PERSONNALISER.md](#)). **Pro** (ou vous) : recherche d'antériorité de la marque (`legal/21_marque-pi.md`). **Vérification** : `STUDIO_STATE.md` à jour.

D. Fondations (P0) — 2-3 semaines

IA crée le projet Unreal C++ (`game/`), les modules `TemporalCore` et `TemporalValley`, la CI Windows, les décisions d'architecture (ADR). **Vous** créez le dépôt **privé** pour le contenu binaire (`game/Content/`), installez un runner GitHub Windows si vous voulez la CI du jeu. **Vérification** : compilation verte, tests automatiques verts.

E. Prototype du fun (P1) — 4-8 semaines · Porte G1

IA code le cœur temporel (tests d'abord) puis un prototype jouable : planter une graine en l'an 0 fait pousser un arbre en l'an 300. **IA** fait jouer des bots et s'auto-évalue sur les piliers de design, puis place dans `QUESTIONS.md` un playtest humain (non bloquant en mode autonome). **Vous** organisez ce playtest avec 5 personnes (kit fourni : `legal/14_playtest-consentement.md`, grille dans `docs/design/51`). **Décision** : si personne ne rit ni ne s'étonne en 10 minutes, on itère ; deux échecs = pivot.

F. Multijoueur (P2) — 6-10 semaines · Porte G2

IA : sessions Steam (AppID de test), 4 joueurs, 4 époques, fantômes, contrôle d'intégrité. **Vous** : session de test à 4 avec des amis. **Vérification** : test Gauntlet « 4 hash identiques ».

G. Boucle de partie (P3) — 8-12 semaines · Porte G3

IA : manches, pause café, rotation, contrats, paradoxes, dangers, HUD. **Vous** : playtest de 8 personnes ; dépôt de la marque ; ouverture de la page Steam (Steam Direct) ; commande de la capsule. **Vérification** : note « je rejouerais » ≥ 4/5.

H. Monde réaliste et contenu (P4) — 6-12 mois

IA prépare la **liste d'achats** d'assets (Fab, MetaHuman) avec licences et coûts. **Vous** achetez et ajoutez les assets ; commandez les créatures à un artiste ; validez la direction artistique. **IA** construit le paysage, les époques, la végétation procédurale, les prefabs de vieillissement, mesure les performances. **Vérification** : 10 graines de monde jouables ; budgets de performance tenus sur les 3 profils.

I. Machine à clips (P5) — 6-10 semaines

Musée de fin de partie, capsules partageables, mode solo, tutoriel, mode photo.

J. Voix, streamer, saboteur (P6) — 6-8 semaines · Porte G4 (alpha)

IA : voix inter-époques, outils anti-harcèlement, mode streamer (désactivé par défaut). **Vérification** : test « aucun audio sur disque », 3 streamers en test fermé.

K. Conformité (P7) — 8-12 semaines · Porte G5 (bêta)

IA met à jour le corpus `legal/` d'après le jeu réel et prépare le dossier pour l'avocat. **Pro** : relecture juridique. **Vous** : intégrez les corrections. **IA + vous** : accessibilité complète, localisation, pseudo-localisation.

L. Démo et Steam (P8) — 8-12 semaines · Porte G6

Build démo, succès, classements, Cloud, profilage, build Shipping. Questionnaire de contenu Steam (dont la section IA).

M. Médias — en continu dès G3

IA rend les plans de `data/shotlist.json` **dans Unreal** (Movie Render Graph), monte les trailers (Sequencer + Remotion). **Vous** validez chaque image dans `media/APPROVALS.md`. Aucune image non validée n'est publiée.

N. Landing page — 1-2 semaines, puis mises à jour

IA construit la landing **cinématique** (`docs/design/34_LANDING_CINEMATIQUE.md`) : mode typographique animé tant qu'aucun rendu n'est validé, puis bascule automatique vers le héros vidéo et la séquence des 4 époques au défilement dès que les plans L00-L04 sont validés ; elle lance les tests (Robot Framework, Lighthouse, k6, OWASP ZAP). **Vous** validez les textes et déclenchez la publication (workflow manuel `pages.yml`).

O. Lancement — 4-8 semaines

Steam Next Fest, programme créateurs (`legal/12_politique-createurs.md`), communication. **Vous** publiez ; l'IA prépare.

Z. Après le lancement

Correctifs sous 72 h pour les problèmes graves, mises à jour régulières, saisons (`docs/design/71_LIVE_OPS.md`).

Pendant les arrêts obligatoires, l'IA prépare tout (liste de clics, fichiers) et continue les chantiers indépendants (landing, juridique, données, Remotion).

Rappel : l'IA ne fait jamais seule un achat, une publication, une signature, ni ne choisit le nom ou le prix. Voir [04_SECURITE_ET_CONFIDENTIALITE.md](#).

Installation du poste de studio

 [English version](#)

1. Matériel

	Minimum pratique	Recommandé
Système	Windows 10 64 bits	Windows 11 64 bits
Processeur	8 cœurs	12-16 cœurs (compilation C++ et shaders)
Mémoire	16 Go (lent)	32-64 Go
Carte graphique	8 Go de VRAM (RTX 2070 / RX 6600 XT)	RTX 3070/4070 ou mieux, 12 Go+
Stockage	300 Go SSD	1 To NVMe
Réseau	fibre conseillée	moteur + assets = 100-200 Go à télécharger

macOS est possible pour Unreal (Epic Games Launcher + Xcode) mais le projet cible d'abord Windows. Linux sert pour les outils annexes (landing, tests).

2. Logiciels

Le skill studio-setup installe tout via winget après votre accord :

```
powershell -ExecutionPolicy Bypass -File .claude/skills/studio-setup/scripts/install_windows.ps1
-Plan # voir le plan
powershell -ExecutionPolicy Bypass -File .claude/skills/studio-setup/scripts/install_windows.ps1
# installer
```

Outil	Pourquoi
Epic Games Launcher + Unreal Engine 5.8	moteur (installation dans le launcher, par vous)
Visual Studio 2022 (jeux C++ + bureau C++) ou Rider	compilation
.NET 8, Git, Git LFS, GitHub CLI, Python 3.12	outils de build et du kit
Node LTS, ffmpeg	landing page, vidéo
Blender LTS	retouches d'assets
Robot Framework, k6, gitleaks, semgrep	tests et sécurité

3. Brancher Claude Code sur Unreal (plugin officiel Epic)

1. Le plugin est déjà déclaré dans `.claude/settings.json` (`enabledPlugins`) : Claude Code propose de l'installer à l'ouverture du dossier. Sinon : `/plugin install unreal-engine-skills-for-claude-code@claude-plugins-official` (et `frontend-design@claude-plugins-official` pour la landing).
2. Dans l'éditeur Unreal : *Edit > Plugins*, activer **Model Context Protocol** et **AllToolsets**, redémarrer.
3. Console de l'éditeur : `ModelContextProtocol.StartServer`

4. Sous Windows, Git Bash (installé avec Git) doit être sur le PATH.
5. Test : demandez à Claude « liste les acteurs de la carte ouverte ».

⚠ Ce plugin donne à l'IA un accès large à l'éditeur, y compris l'exécution de Python. Travaillez toujours sur une branche Git et validez (commit) avant chaque série d'opérations.

4. Variables et dossiers

```
setx UE_ROOT "C:\Program Files\Epic Games\UE_5.8"
```

Créez un dépôt **privé** pour game/Content/ (Git LFS ou Perforce). Ne publiez jamais ce dossier.

5. Vérifier

```
py tools\doctor.py
```

Toutes les lignes utiles doivent être ✓.

Temps, coûts et consommation

 [English version](#)

Estimations d'ordre de grandeur au 7 octobre 2026, pour un jeu coop **3D réaliste** sous Unreal Engine 5.8 tel que décrit dans docs/design/. Elles servent à décider, pas à promettre. Mesurez vos chiffres réels à chaque porte et ajustez.

1. Trois scénarios

	A — Solo + IA	B — Studio réduit	C — Petite équipe
Composition	porteur du projet (temps partiel) + Claude Code + freelances ponctuels	porteur à plein temps + 2-3 freelances réguliers (art 3D, animation, son)	5-8 personnes (prog. gameplay, tech art, 2 artistes, animateur, producteur/QA) + IA
Durée jusqu'à l'Early Access	30-48 mois	18-30 mois	15-24 mois
Budget hors rémunération du porteur	≈ 40 000 – 80 000 €	≈ 150 000 – 300 000 €	≈ 500 000 € – 1,2 M€
Niveau visuel atteignable	réaliste grâce aux assets achetés, animations surtout issues des bibliothèques	réaliste cohérent, créatures et PNJ sur mesure	réaliste soigné, capture de jeu d'acteur, contenu plus riche
Risque principal	durée et périmètre	trésorerie	coût fixe mensuel

2. Postes de dépense (fourchettes)

Poste	Scénario A	Scénario C	Remarques
Poste de travail (si nécessaire)	2 000 – 3 500 €	2 500 € × personnes	GPU RTX 4070+, 64 Go conseillés
Unreal Engine	0 €	0 €	5 % au-delà de 1 M\$ de revenus bruts cumulés
Assets Fab / Megascans	1 000 – 6 000 €	8 000 – 30 000 €	Megascans payants depuis 2025
Créatures (sculpt, groom, rig)	3 000 – 10 000 €	15 000 – 40 000 €	Bouloche, chèvres, Chronomites
Capture faciale / corporelle	0 – 3 000 €	15 000 – 60 000 €	MetaHuman Animator (vidéo) vs studio de mocap
Musique (6 pistes + stingers)	3 000 – 8 000 €	10 000 – 30 000 €	cession de droits incluse
Design sonore complémentaire	1 000 – 4 000 €	8 000 – 25 000 €	
Capsule Steam, logo, illustrations	1 500 – 4 000 €	5 000 – 15 000 €	
Montage et étalonnage du trailer	1 000 – 3 000 €	5 000 – 20 000 €	
Localisation (≈ 15 000 mots × 7 langues)	8 000 – 16 000 €	12 000 – 25 000 €	tarifs professionnels au mot ; relecture native
Juridique (CGU, confidentialité, contrats, AIPD)	2 000 – 6 000 €	8 000 – 20 000 €	
Marque (UE, 1-3 classes)	850 – 1 500 €	2 000 – 5 000 €	+ international si besoin
Expert-comptable, assurance RC pro	1 500 – 3 500 €/an	4 000 – 10 000 €/an	
Steam Direct	≈ 100 \$	≈ 100 \$	par jeu
Playtests et QA externe	500 – 3 000 €	10 000 – 40 000 €	
Marketing (salons, relations presse, publicité)	2 000 – 10 000 €	30 000 – 150 000 €	
Abonnement à l'assistant IA	selon l'offre choisie	selon l'offre × personnes	voir les offres sur support.claude.com
Salaires de l'équipe	—	400 000 € – 900 000 €	principal poste du scénario C

Aides possibles en France à étudier : crédit d'impôt jeux vidéo (conditions strictes), fonds d'aide au jeu vidéo du CNC, Bpifrance, aides régionales ([legal/20_fiscalite-redevances.md](#)).

3. Calendrier par phase (scénario B)

Phase	Durée	Porte	Ce qui se passe
A-C Diagnostic, installation, personnalisation	1-2 semaines	—	poste prêt, titre choisi
P0 Fondations	2-3 semaines	—	projet Unreal, CI, ADR
P1 Prototypage du fun	4-8 semaines	G1	graine → arbre jouable, 5 testeurs
P2 Multijoueur	6-10 semaines	G2	4 joueurs en ligne sans désynchronisation
P3 Boucle de partie	8-12 semaines	G3	partie complète, page Steam, premiers clips
P4 Monde réaliste et contenu	6-12 mois	—	le plus long : assets, PCG, prefabs, MetaHumans
P5 Machine à clips	6-10 semaines	—	musée, capsules, solo
P6 Voix, streamer, saboteur	6-8 semaines	G4	alpha
P7 Conformité, accessibilité, localisation	8-12 semaines	G5	bêta, relecture juridique
P8 Démo et Early Access	8-12 semaines	G6	démo, Next Fest, lancement

4. Consommation de l'IA (ordre de grandeur)

- **Temps d'agent** : une phase de code représente typiquement de 20 à 80 sessions de travail de Claude Code, de 30 minutes à quelques heures chacune ; la phase P4 est la plus gourmande (nombreuses opérations dans l'éditeur via MCP).
- **Jetons** : très variable selon la taille du projet, la mise en cache et le nombre d'opérations d'éditeur. Comptez plusieurs millions de jetons par journée de travail intensive. Mesurez votre consommation réelle (commande de coût/usage de votre outil) dès la phase P0 et reportez-la dans `STUDIO_STATE.md` pour extrapoler.
- **Leviers d'économie** : une phase par session, mode plan d'abord, petites PR, tests ciblés, éviter de relire tout le dépôt à chaque session (le skill `game-studio` lit `STUDIO_STATE.md` en premier).
- **Temps humain incompressible** : revues de PR ($\approx 20\text{-}30\%$ du temps d'agent), playtests, achats d'assets, direction artistique, validation des médias et des textes, décisions de porte.

5. Ce qui fait exploser les estimations

Le périmètre (ajouter une époque, un biome, un mode), la qualité des humains réalistes, la capture de mouvement, la localisation tardive, et l'absence de playtests tôt. La règle de coupe de `docs/design/50_PRODUCTION_PLAN.md` §6 s'applique avant toute rallonge budgétaire.

Sécurité et confidentialité : comment le kit vous protège

🇬🇧 [English version](#)

Modèle de sécurité du kit

L'IA peut

- ✓ lire et écrire dans le dépôt
- ✓ installer depuis les sources officielles, après accord
- ✓ compiler, tester, mesurer
- ✓ piloter l'éditeur Unreal sur une branche
- ✓ préparer documents, médias, landing page
- ✓ proposer achats, textes et décisions

L'IA ne fait jamais seule

- × acheter, créer un compte, saisir un identifiant
- × publier (GitHub public, Steam, réseaux, site)
- × signer, déposer une marque, fixer le prix
- × lire ~/.ssh, .env, trousseaux, fichiers perso
- × ajouter traceur, cookie, CDN, télémétrie
- × publier Content/ ou des assets sous licence

Garde-fous : .claude/settings.json (deny/ask) · tools/privacy_scan.py · hook pré-commit · CI (gitleaks, semgrep, ZAP) · media/APPROVALS.md

Votre liste locale de termes interdits (~/.config/300yl/denylist.txt) n'est jamais versionnée.

1. Ce que l'IA peut faire seule

Lire et écrire **dans le dépôt**, installer des outils **depuis les sources officielles après votre accord**, compiler, tester, piloter l'éditeur Unreal via MCP sur une branche, préparer des documents et des médias.

2. Ce que l'IA ne fait jamais seule

Action	Pourquoi
Acheter (assets, Steam Direct, prestataires)	votre argent
Créer un compte, saisir un identifiant	votre identité
Publier (GitHub public, Steam, réseaux, site)	votre responsabilité d'éditeur
Signer, déposer une marque	engagement juridique
Choisir le titre, le prix, les textes publics	décisions d'éditeur
Lire vos fichiers personnels, clés SSH, .env, trousseaux	règles deny de .claude/settings.json
Envoyer vos données vers un service tiers	aucun appel réseau hors sources officielles
Ajouter un traceur, un cookie, un CDN	interdit et détecté par privacy_scan.py
Publier game/Content/ ou des assets Epic/Fab/MetaHuman	interdit par les licences, bloqué par la CI

3. Protéger vos données personnelles

1. Créez ~/.config/300yl/denylist.txt (hors du dépôt) avec vos nom, prénom, e-mail, pseudo, employeur, ville. tools/privacy_scan.py refusera tout fichier qui les contient.

2. Configurez Git avec l'adresse anonyme GitHub : `git config user.email "<id>+<pseudo>@users.noreply.github.com"`.
3. Activez le contrôle avant commit : `git config core.hooksPath .githooks`.
4. Utilisez une adresse e-mail dédiée au projet pour tous les textes publics (CGU, mentions légales, presse).
5. Supprimez les métadonnées des médias avant publication (`exiftool -all=`), ce que fait le skill `privacy-guard`.

4. Vérifier vous-même

```
python3 tools/privacy_scan.py
cat .claude/settings.json
git ls-files | grep -Ei '\.(uasset|umap|pak)$|^game/Content/' # doit être vide
```

5. Les joueurs

Le jeu applique la protection par défaut (`legal/03`, `legal/09`) : voix jamais enregistrée, statistiques en opt-in, parties entre amis par défaut, aucun achat intégré.

Personnaliser le projet (ou en faire votre propre jeu)

 [English version](#)

Tout est conçu pour être modifié. Licence : code MIT, contenus CC BY 4.0 (créditez « The 300 Years Later contributors »). Le titre et la marque ne sont pas couverts par la licence.

0. Dire à l'IA ce que vous voulez

- **Une fois pour toutes** : `extra_instructions` dans `studio.config.yaml` (ex. « anglais d'abord, Steam Deck vérifié prioritaire, pas de chat vocal »).
- **Pour une session** : ajoutez vos consignes après le prompt de démarrage. Elles l'emportent sur le fichier ; seul le contrat de sécurité ne peut pas être levé.

1. Changer le titre

1. Choisissez un titre et faites la recherche de marque (`legal/21_marque-pi.md`).
2. Demandez à l'IA : « Remplace le titre public "300 Years Later" par "" partout où il est public (README, landing, Remotion, docs/design/70), sans toucher au nom de code CENTURY TEMPS ni aux identifiants techniques. »
3. Vérifiez : `python3 tools/privacy_scan.py` et les tests de la landing.

2. Modifier le jeu sans coder

Vous voulez...	Modifiez	Vérifiez
ajouter une transformation dans le temps	<code>data/recipes/*.json</code> (format : <code>data/schemas/recipe.schema.json</code>) + un modèle de Chronique dans <code>data/phrases/</code>	<code>python3 tools/validate_data.py</code>
ajouter un contrat	<code>data/contracts/*.json</code>	idem
régler la durée des manches, les vitesses, le score	<code>data/tuning.json</code>	playtest
changer le look d'une époque	<code>docs/design/33_VISUAL_TARGETS.md</code> §2 puis demander à l'IA d'appliquer	revue visuelle
ajouter un plan de trailer	<code>data/shotlist.json</code>	rendu + validation

3. Réutiliser le studio pour un autre jeu

Les skills (`.claude/skills/`) sont génériques à 80 % : remplacez `docs/design/` par votre propre conception (en gardant la même structure de fichiers), adaptez `CLAUDE.md`, puis lancez `game-studio`. Gardez le contrat de sécurité, les portes de décision et le corpus juridique comme base.

Utiliser le kit avec d'autres IA

 [English version](#)

Outil	Comment
Claude Code	natif : skills dans <code>.claude/skills/</code> , règles dans <code>CLAUDE.md</code> , permissions dans <code>.claude/settings.json</code> , plugin officiel Epic pour Unreal
Gemini CLI	lit <code>GEMINI.md</code> , qui importe <code>AGENTS.md</code>
GitHub Copilot	lit <code>.github/copilot-instructions.md</code> , qui renvoie à <code>AGENTS.md</code>
Kimi, Codex, Aider, Cursor, Windsurf, autres	lisent <code>AGENTS.md</code> (sinon : « Lis <code>AGENTS.md</code> » en premier message), qui pointe vers les skills comme procédures Markdown
Agent sans accès à l'éditeur Unreal	Python d'éditeur en ligne de commande : <code>UnrealEditor-Cmd.exe <projet> -run=pythonscript -script=game/Scripts/<script>.py</code> et compilation via <code>Build.bat</code>

Prompt universel :

Lis `AGENTS.md` puis applique le skill `game-studio`. Travaille en autonomie de A à Z selon `studio.config.yaml` ; ne me sollicite qu'aux arrêts obligatoires, en regroupant tes questions avec ta recommandation. Réponds-moi en français. *(facultatif)* Instructions en plus : ...

Quel que soit l'outil : reproduisez les règles deny de `.claude/settings.json` dans sa configuration de permissions, et lancez `tools/privacy_scan.py` avant chaque commit.

Si l'outil n'a pas de skill `frontend-design`, la landing suit strictement `docs/design/34_LANDING_CINEMATIQUE.md` et `30_ART_BIBLE.md`. Si l'outil ne gère pas les plugins, l'installation du plugin Epic se fait côté Claude Code ou est remplacée par le Python d'éditeur.

FAQ et dépannage

 [English version](#)

Le dépôt contient-il des images du jeu ? Non, volontairement. Le jeu n'existe pas encore : toutes les images seront rendues dans Unreal Engine selon `docs/design/33_VISUAL_TARGETS.md` et `data/shotlist.json`, puis validées par un humain. Les schémas du dépôt expliquent le fonctionnement, ils ne montrent pas le jeu.

L'IA fait-elle vraiment tout toute seule ? Presque tout : installation, code, tests, rendus, landing, documents. Elle s'arrête seulement là où la loi, les licences ou votre argent l'exigent : créer vos comptes, installer Unreal depuis le launcher (connexion Epic), acheter, signer, publier, valider les médias, fixer le prix. Elle prépare tout pour que ce soit l'affaire de quelques clics, et continue les autres chantiers pendant ce temps. Un jeu de cette ambition demande aussi des humains (playtesteurs, artistes pour les créatures, compositeur, juriste).

Puis-je ajouter mes propres consignes ? Oui : dans votre message (prioritaire) ou dans `extra_instructions` de `studio.config.yaml`. Seul le contrat de sécurité ne peut pas être levé.

L'IA peut-elle faire un jeu du niveau de GTA ? Non. Ces productions mobilisent des centaines de personnes et des budgets de centaines de millions. Le kit vise un jeu indépendant réaliste de très haute qualité, avec des professionnels pour l'art organique (créatures, jeu d'acteur, musique). Voir `docs/guides/03_TEMPS_ET_COUTS.md`.

Combien ça coûte ? De $\approx 40\,000$ € (solo + IA) à plus de $500\,000$ € (petite équipe), hors salaire du porteur. Détails dans le guide 03.

Puis-je vendre le jeu que je crée avec ce kit ? Oui (licences MIT et CC BY 4.0, avec attribution). Respectez le CLUF Unreal, les licences Fab/MetaHuman, et choisissez votre propre titre.

Le serveur MCP Unreal ne répond pas. L'éditeur doit être ouvert, les plugins Model Context Protocol et AllToolsets activés, le serveur démarré (`ModelContextProtocol.StartServer`), et Git Bash sur le PATH sous Windows.

La compilation est très lente. Exclure les dossiers du projet de l'antivirus (avec prudence), activer le cache DDC partagé, ajouter de la RAM. Compiler en `Development Editor` pendant le développement.

`privacy_scan.py` bloque mon commit. Il a trouvé un e-mail, un numéro, un jeton, un traceur ou un terme de votre liste locale. Corrigez le fichier indiqué ; ne contournez pas le contrôle.

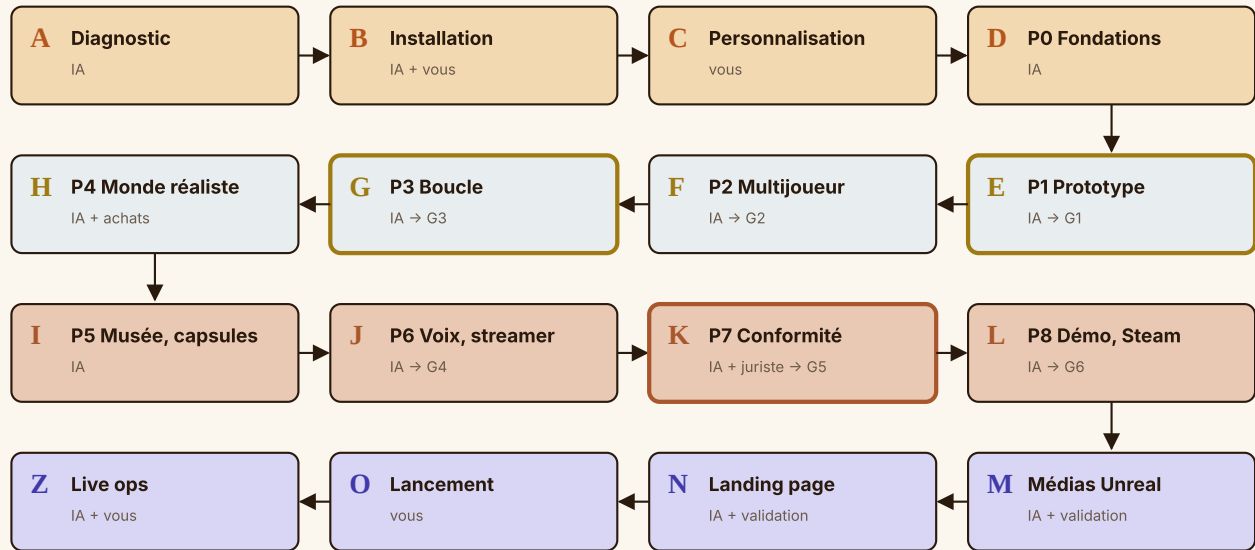
Les tests Robot Framework échouent localement. Lancez `rfbrowser init` une fois après l'installation (télécharge les navigateurs de test).

Puis-je utiliser des images générées par IA pour aller plus vite ? Le kit l'interdit par défaut : risque juridique (droits), perception des joueurs, obligation de déclaration Steam. Si vous le décidez malgré tout, consignez-le dans un ADR, déclarez-le sur Steam et dans `legal/17_transparence-ia.md`.

Schémas

Le parcours du studio, de A à Z

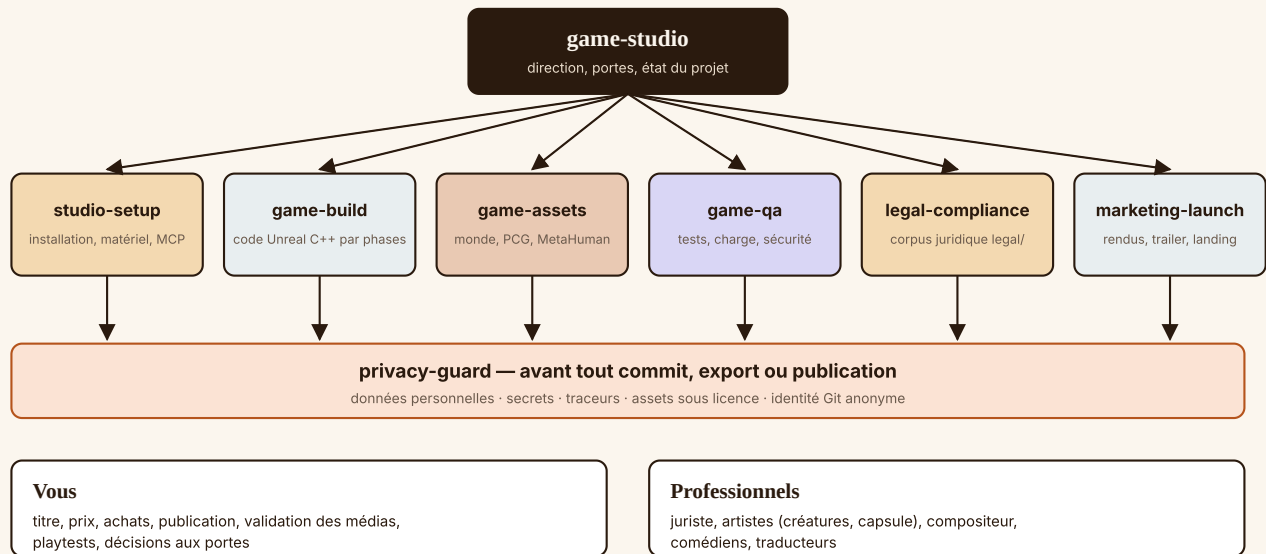
Les portes en couleur sont des arrêts obligatoires : playtest ou décision humaine.



parcours a z

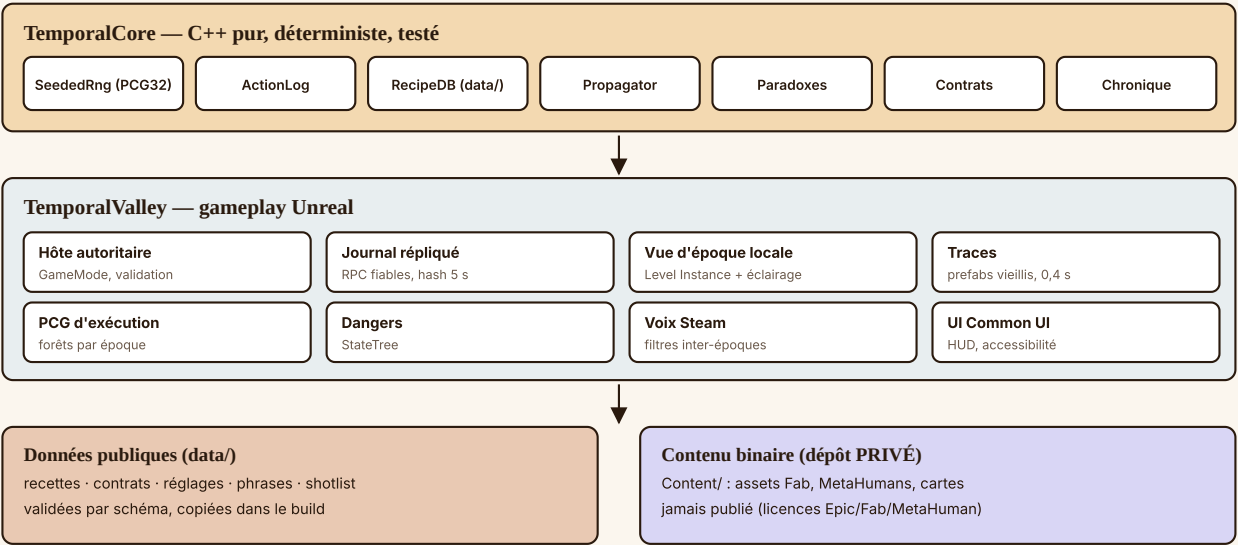
Les skills : un studio en équipes

game-studio dirige ; chaque équipe a son skill ; privacy-guard contrôle tout ce qui sort.



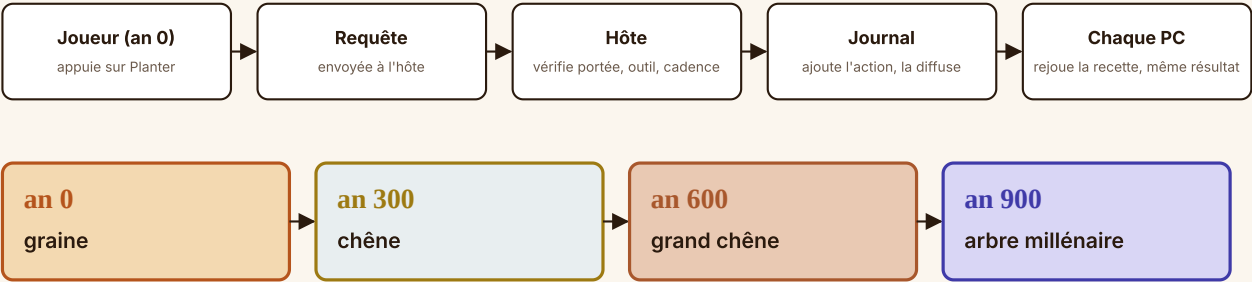
orchestration skills

Architecture du jeu (Unreal Engine 5.8)



architecture unreal

Ce qui se passe quand un joueur plante une graine



La graine de monde + le journal + les recettes suffisent à reconstruire les 4 époques : sauvegarde, capsule et musée utilisent le même journal.
Si l'an 300 avait modifié l'arbre et que l'an 0 arrache la graine : non-conformité, la jauge monte, les Chronomites arrivent.

propagation

Modèle de sécurité du kit

L'IA peut

- ✓ lire et écrire dans le dépôt
- ✓ installer depuis les sources officielles, après accord
- ✓ compiler, tester, mesurer
- ✓ piloter l'éditeur Unreal sur une branche
- ✓ préparer documents, médias, landing page
- ✓ proposer achats, textes et décisions

L'IA ne fait jamais seule

- × acheter, créer un compte, saisir un identifiant
- × publier (GitHub public, Steam, réseaux, site)
- × signer, déposer une marque, fixer le prix
- × lire ~/.ssh, .env, trousseaux, fichiers perso
- × ajouter traceur, cookie, CDN, télémétrie
- × publier Content/ ou des assets sous licence

Garde-fous : `.claude/settings.json (deny/ask)` · `tools/privacy_scan.py` · hook pré-commit · CI (`gitleaks`, `semgrep`, `ZAP`) · `media/APPROVALS.md`

Votre liste locale de termes interdits (`~/config/300yl/denylist.txt`) n'est jamais versionnée.

modele securite

Captures d'écran

Captures réelles réalisées lors de la génération de ce guide.

Diagnostic de l'environnement (étape A)

```
$ python3 tools/doctor.py
Système : Linux 6.18.44-fc-v77 – Python 3.13.16

✓ git      Versionnage      [tout] git version 2.43.0
✓ git-lfs  Contenu binaire (dépôt privé) [game-build] git-lfs/3.4.1 (GitHub; linux amd64; go 1.22.2)
✓ python3  Outils du kit      [tout] Python 3.13.16
x dotnet   .NET (UnrealBuildTool)      [game-build] MANQUANT → winget install Microsoft.DotNet.SDK.8
✓ node     Landing page / Remotion      [marketing-launch] v22.22.0
✓ npm      Paquets web          [marketing-launch] 10.9.4
✓ ffmpeg   Encodage vidéo, GIF      [marketing-launch] ffmpeg version 6.1.1-3ubuntu5 Copyright (c) 2000-2023 the FF
x blender  Retouches d'assets      [game-assets] MANQUANT → https://www.blender.org/download/lts/
x robot    Tests E2E Robot Framework [game-qa] MANQUANT → pip install robotframework robotframework-browser
x k6       Tests de charge web      [game-qa] MANQUANT → winget install GrafanaLabs.k6
x gitleaks Détection de secrets      [game-qa] MANQUANT → winget install Gitleaks.Gitleaks
x semgrep  Analyse statique sécurité [game-qa] MANQUANT → pip install semgrep
✓ gh       Publication GitHub (par l'humain) [publication] gh version 2.89.0 (2026-03-26)
x unreal   Unreal Engine 5.8        [game-build] MANQUANT → Epic Games Launcher (voir studio-setup)

Les outils manquants sont installés par le skill « studio-setup ».
```

Diagnostic de l'environnement (étape A)

Validation des données du jeu

```
$ python3 tools/validate_data.py
[validate_data] 25 recettes, 12 contrats, 16 modèles – jsonschema complet
✓ données valides
```

Validation des données du jeu

Scan de confidentialité avant publication

```
$ python3 tools/privacy_scan.py
[privacy_scan] 178 fichiers, 0 termes locaux, 0 problème(s)
  i Astuce : créez ~/.config/300yl/denylist.txt avec vos nom, e-mail, pseudo, employeur.
```

Scan de confidentialité avant publication

Audit des licences

```
$ python3 tools/license_audit.py
[license_audit] 0 dossier(s) vérifié(s), 0 problème(s)
```

Audit des licences

L'Aube

an 0

Tu plantes une graine. Ton ami la retrouve trois cents ans plus tard, devenue chêne.

Un jeu coopératif pour 1 à 4 joueurs. Chacun vit dans un siècle différent de la même vallée, et tout ce que vous laissez derrière vous vieillit sous les yeux des autres.

[Ajouter à ma liste de souhaits](#)[Comment ça marche](#)

Landing page de référence — bureau (mode typographique)

an 300

Les Bannières

Ta graine est un chêne. Le bourg a construit son marché dessous. Ton amie y attache une balançoire.

an 600

La Vapeur

Le chêne est géant. L'usine voisine l'a épargné, on ne sait pas pourquoi. Les ouvriers déjeunent à son ombre.

an 900

Le Néon

C'est l'Arbre millénaire, couvert de lanternes. Le musée raconte, très sérieusement, que tu l'as planté « pour faire de l'ombre à personne ».

Quatre amis, quatre

Vous jouez au même endroit, mais pas à la même époque. Tu vois tes amis comme des silhouettes translucides et tu les entends de plus en plus étouffés à mesure que vous

Landing — la page prend la couleur de l'époque lue

L'Aube

an **O**

Tu plantes une graine. Ton ami la retrouve trois cents ans plus tard, devenue chêne.

Un jeu coopératif pour 1 à 4 joueurs.
Chacun vit dans un siècle différent de la même vallée, et tout ce que vous laissez derrière vous vieillit sous les yeux des autres.

Ajouter à ma liste de souhaits

Comment ça marche

Images du jeu

Aucun rendu validé à ce jour. Les plans à produire sont décrits dans docs/design/33_VISUAL_TARGETS.md, docs/design/34_LANDING_CINEMATIQUE.md et data/shotlist.json ; ils apparaîtront ici automatiquement une fois marqués « validé » dans media/APPROVALS.md.